

Programming Techniques

MIT – Sabaragamuwa University of Sri Lanka

Week 5 — Arrays, Structures & Git Branching

 2D Arrays

 Structures

 Git Branching

 Conflicts

Lecturer: Kalinga Gunawardhana

Faculty of Graduate Studies · Sabaragamuwa University of Sri Lanka

Session Structure — 3 Hours

Part A: 2D Arrays (40 min)

Part B: Structures (40 min)

Part C: Git Branching (40 min)

Part D: Conflicts (40 min)

Session Roadmap



Part A

2D Arrays

Declare · Access
Traverse · Sum



Part B

Structures

Declare · Create
Arrays of Structs



Part C

Git Branching

Branch · Switch
Merge



Part D

Conflicts

Conflict markers
Resolve · Commit

Each part = ~25 min concept + 15 min hands-on practical | Total: 3 hours

What is a 2D Array?

2D Array: A collection of values arranged in rows and columns — like a spreadsheet or table.

	Col 0	Col 1	Col 2
Row 0	1	2	3
Row 1	4	5	6
Row 2	7	8	9

`matrix[1][2] = 6`

```
matrix = [[1, 2, 3],  
          [4, 5, 6],  
          [7, 8, 9]]  
  
rows = len(matrix)      # → 3  
cols = len(matrix[0])   # → 3
```



Cinema Seats

Row 2, Seat 4
= `seats[2][4]`



Spreadsheet

Row = record
Col = field



Chess Board

`board[row][col]`
= piece

Declaring a 2D Array — 3 Stages

Stage 1 — Empty grid (all zeros)

```
matrix = [[0, 0, 0],  
          [0, 0, 0],  
          [0, 0, 0]]
```

Stage 2 — With actual values

```
matrix = [[1, 2, 3],  
          [4, 5, 6],  
          [7, 8, 9]]
```

Stage 3 — Measure size

```
rows = len(matrix)      # → 3  
cols = len(matrix[0])   # → 3
```

matrix[0] → first row | **matrix[0][0]** → value at row 0, col 0 | **len(matrix[0])** → number of columns

Accessing Individual Cells

Syntax: `matrix[row][col]` — row index FIRST, column index SECOND. Always.

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

# Reading a value
print(matrix[1][2])    # row 1, col 2 → 6

# Changing a value
matrix[0][1] = 99
print(matrix[0][1])    # → 99

# What NOT to do
print(matrix[3][0])    # ❌ IndexError – row 3 does not exist!
```

1	2	3
4	5	6
7	8	9

`matrix[1][2] = 6`



Common Mistake

Swapping row and column — `matrix[col][row]` — gives wrong data or causes a crash. Row is always first.

What is Traversal?

Traversal: Visiting every element in a data structure — one by one, in order, without skipping any.

1	2	3	→→→ Row 0
4	5	6	→→→ Row 1
7	8	9	→→→ Row 2



Then next row



Reading a book

Every word, left to right, line by line



Cinema check

Every seat in row 1, then row 2, then row 3



Attendance register

Every name, top to bottom, none skipped

? Why do we traverse?

To search for a value, sum all elements, compare, print, or modify every cell — any operation that must touch each value.

For a 2D array, traversal visits every [row][col] cell — we need TWO loops to do this.

Traversal — Building the Nested Loop

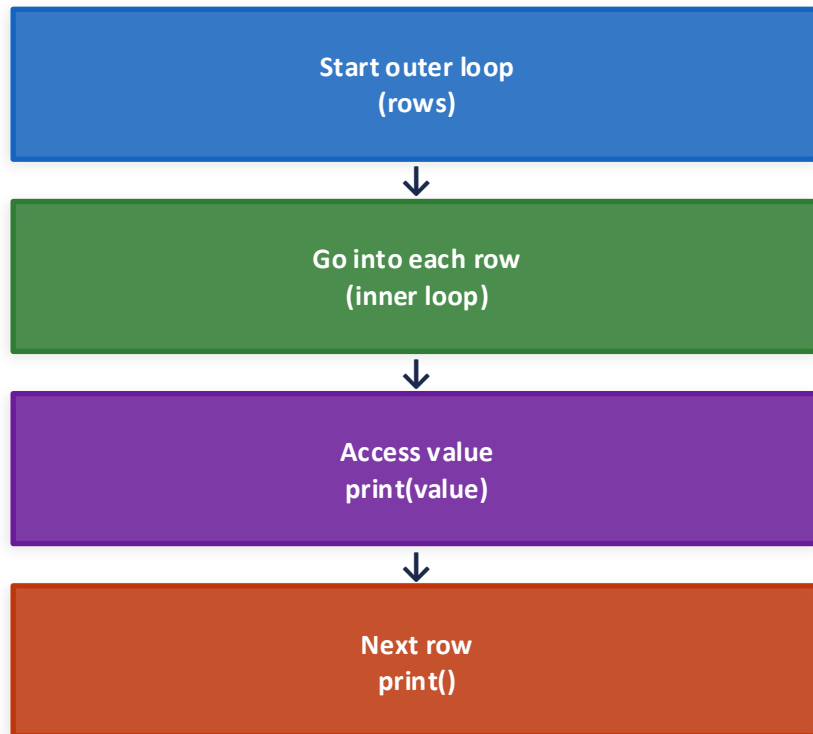
Step 1 — Loop through rows (outer loop):

```
for row in matrix:  
    print(row)  
# Output:  
# [1, 2, 3]  
# [4, 5, 6]  
# [7, 8, 9]
```

Step 2 — Add inner loop to reach each cell:

```
for row in matrix:  
    for value in row:      # ← inner loop  
        print(value, end=" ")  
    print()                # new line after each row  
# Output:  
# 1 2 3  
# 4 5 6  
# 7 8 9
```

`end=" "` keeps values on the same line | `print()` moves to the next row



Traversal — Index-Based Loop

When to use: When you need to know WHERE you are (the coordinates), not just WHAT the value is.

```
# Index-based loop – gives you the exact coordinates
for r in range(len(matrix)):
    for c in range(len(matrix[r])):
        print(f"[{r}][{c}] = {matrix[r][c]}")
```

Output:

```
# [0][0] = 1   [0][1] = 2   [0][2] = 3
# [1][0] = 4   [1][1] = 5   [1][2] = 6
# [2][0] = 7   [2][1] = 8   [2][2] = 9
```

Practical use — sum all elements:

```
total = 0
for r in range(len(matrix)):
    for c in range(len(matrix[r])):
        total += matrix[r][c]
print("Sum:", total)    # → 45
```



Search

Index loops let you record which `[r][c]` holds the target value



Modify

Change specific cells — `matrix[r][c] = new_value`



Sum / Compare

Accumulate or compare values across all cells

Rule: use `for value in row` when you only need the value. Use `range(len(...))` when you need the index.



Practical 1 — Matrix Calculator



15 min

Step 1



Declare the matrix

Create a 3×3 matrix:
`matrix = [[2,4,6],[1,3,5],[7,8,9]]`



Step 2



Print as a grid

Loop through rows and columns
Print each row on its own line



Step 3



Calculate the sum

Use a nested loop
Add every element to a total variable

Expected output:

1 2 3

4 5 6

7 8 9

Sum: 45



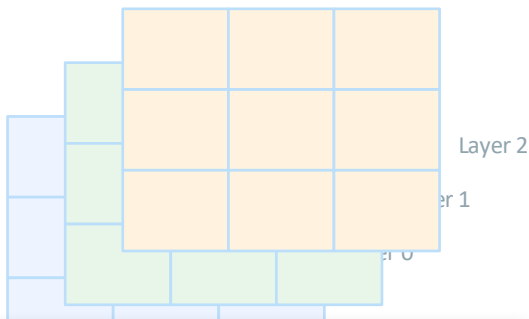
Bonus: Find and print the largest value in the matrix



Online: Paste your output in the chat when done

3D Arrays — One Step Further

3D Array: Like multiple "pages" of a 2D spreadsheet — indexed as `cube[layer][row][col]`



```
# Declare a 3D array: 2 layers, 3 rows, 4 columns
cube = [[[0]*4 for _ in range(3)] for _ in range(2)]

# Access an element
cube[1][2][3] = 99      # layer 1, row 2, col 3

# Navigate: layer → row → col
for layer in cube:
    for row in layer:
        for val in row:
            print(val, end=" ")
```



Image pixels

`image[height][width][RGB]`



Game grid

`grid[floor][row][col]`



3D data

`data[time][x][y]`

Note: 2D arrays are most common. 3D arrays appear in image processing and simulations — conceptual awareness is sufficient for now.

What is a Structure?

Structure: A custom data type that groups related fields of different types under one name — like a labelled form.



Groups related data

name, age, and GPA all belong to one Student — kept together



You define the type

Design your own blueprint once, then create as many objects as you need



Dot notation access

Use `object.field` to read or change any field — e.g. `s1.name`



Array vs Structure

Array = same type repeated. Structure = mixed types in one container

Why We Need Structures — The Problem

✗ Without a Structure

```
# Messy separate variables
student_name = "Amal"
student_age  = 25
student_gpa  = 3.8

# Now add a second student...
student_name_2 = "Nimal"
student_age_2  = 27
student_gpa_2  = 3.5

# 30 students = 90 variables ✗
```

Unmanageable as data grows!

✓ With a Structure

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    age: int
    gpa: float

# One object holds all data
s1 = Student("Amal", 25, 3.8)
s2 = Student("Nimal", 27, 3.5)
# Clean, scalable ✓
```

Define once — use for every student.

A structure is a blueprint — define it once, create as many objects as needed.

Anatomy of a Structure Declaration

```
from dataclasses import dataclass    # ← import the tool

@dataclass                          # ← decorator: marks this as a struct
class Student:                      # ← name of your new type (capitalised)
    name: str                       # ← field 1: stores text
    age: int                        # ← field 2: stores a whole number
    gpa: float                      # ← field 3: stores a decimal
```

@dataclass

Tells Python this class is a data container — generates `__init__` automatically

class Student

You are defining a brand-new data type called Student

name: str

A field named 'name' that holds text (string)

age: int

A field named 'age' that holds a whole number (integer)

gpa: float

Student

name
(str)

age
(int)

gpa
(float)



Field order matters — when you create an object, values must be given in the same order as the fields.

Creating Structure Objects

```
# Step 1 – Create one student object
s1 = Student("Amal", 25, 3.8)
#           ↑      ↑      ↑
#           name   age   gpa

# Step 2 – Create another
s2 = Student("Nimal", 27, 3.5)

# Step 3 – Print the whole object
print(s1)
# Output: Student(name='Amal', age=25, gpa=3.8)
```

s1 : Student

name : "Amal"

age : 25

gpa : 3.8

Rule: Values go in the SAME ORDER as the field declarations in the class.

 **Common Mistake — Wrong order!**

```
s3 = Student(25, "Amal", 3.8) # ❌ age gets "Amal" – wrong type!
```

Accessing & Modifying Fields

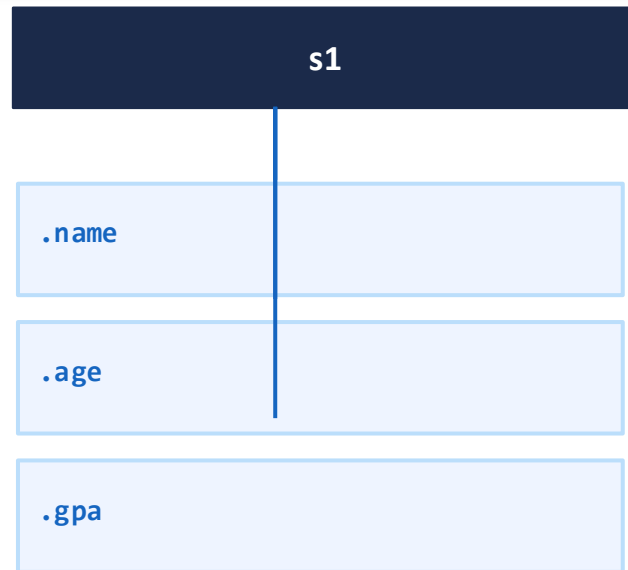
object.field — the dot (.) is the 'belongs to' operator. It connects the object to its field.

```
s1 = Student("Amal", 25, 3.8)

# Reading individual fields
print(s1.name)    # → Amal
print(s1.age)     # → 25
print(s1.gpa)     # → 3.8

# Using a field in a condition
if s1.gpa >= 3.5:
    print(s1.name, "is on the Dean's List")
# → Amal is on the Dean's List

# Modifying a field
s1.gpa = 3.9
print(s1.gpa)     # → 3.9
```



You can also compare: `s1.gpa > s2.gpa` — this works just like comparing plain numbers.

Arrays of Structures — Combining Both Concepts

```
# Stage 1 — Build the list
students = [
    Student("Amal", 25, 3.8),
    Student("Nimal", 27, 3.5),
    Student("Kamala", 24, 3.9),
    Student("Saman", 26, 3.2),
]

# Stage 2 — Access by index then field
print(students[0].name)    # → Amal
print(students[2].gpa)     # → 3.9

# Stage 3 — Loop through all records
for student in students:
    print(f"{student.name:10} | Age: {student.age} | GPA: {student.gpa}")
```

Student	
.name: Amal	.gpa: 3.8

Student	
.name: Nimal	.gpa: 3.5

Student	
.name: Kamala	.gpa: 3.9

Student	
.name: Saman	.gpa: 3.2

Key syntax: `students[2].gpa` — index into the array FIRST to get the object, then use dot notation to access the field.

Array of structs = the foundation of every database table — each row is one record.



Practical 2 — Student Record System



15 min

Step 1



Define the dataclass

Create Student with
name (str), age (int)
gpa (float)

Step 2



Build the list

Create a list of
5 Student objects
with real values

Step 3



Print all records

Loop and print each
student on one line
formatted neatly

Step 4



Find highest GPA

Loop to compare
gpa values and
print the winner



Bonus: Sort students by GPA descending and print the ranked list



Online: Share the name + GPA of your top student in the chat

Part A Recap — Arrays & Structures

2D Array

Grid of same-type values. Access with `matrix[row][col]`. Row index always first.

Traversal

Visit every cell using nested loops. Value-based or index-based depending on the task.

Structure

Groups mixed-type fields under one name. Define once with `@dataclass`, reuse for every object.

Arrays of Structs

Combine both — a list of objects. `students[i].field` to access any record's data.

 Up next: Version control for your code — Git Branching

Quick Recap — Git Essentials (Week 4)

git init

Start repo

git add

Stage files

git commit

Save snapshot

git push

Send to GitHub

git pull

Get latest

 **commit**

Saves a snapshot of all staged changes with a message

 **push**

Sends your local commits up to the remote GitHub repository

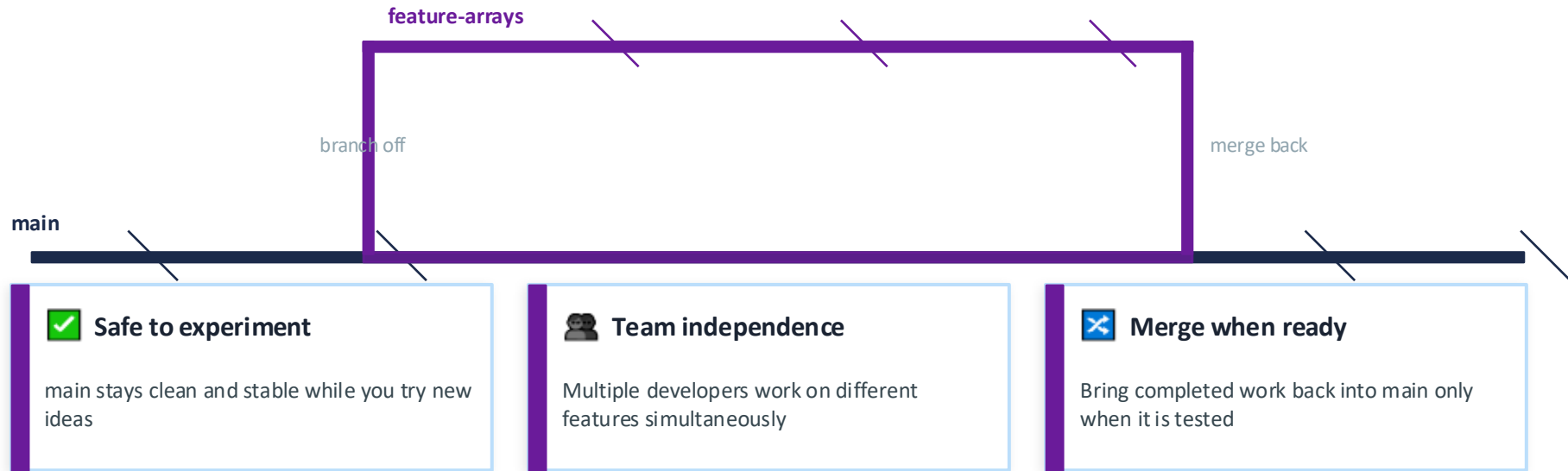
 **pull**

Downloads the latest changes from GitHub to your local machine

Week 4 covered the core workflow. Today we branch off the main timeline — creating parallel histories for safe development.

What is a Branch?

Branch: An independent copy of your code timeline — changes on a branch do NOT affect main until you merge.



Rule: Never work directly on main — always create a branch first.

Branch Commands — Creating & Switching

Step 1 — See which branch you are on (asterisk = current)

```
git branch
```

```
# Output: * main
```

Step 2 — Create a new branch

```
git branch feature-arrays
```

```
git branch
```

```
# Output:      feature-arrays
```

```
#             * main
```

Step 3 — Switch to the new branch

```
git checkout feature-arrays
```

```
# Output: Switched to branch 'feature-arrays'
```

```
git branch
```

```
# Output: * feature-arrays
```

```
#             main
```

Shortcut

```
git checkout -b feature-arrays
```

```
# Creates AND switches in one command
```

```
# Equivalent to: git branch + git checkout
```

The * asterisk

Always shows which branch you are currently working on.
Every commit you make goes to this branch.

Best practice workflow:

```
checkout -b → code → commit → merge
```



Always confirm which branch you are on before committing — run `git branch` first.

Branch Commands — List, Rename & Delete

```
# List local branches
git branch
# Output: * main
#         feature-arrays
#         feature-student

# List all branches including remote
git branch -a

# Rename current branch
git branch -m new-name

# Delete a merged branch (safe)
git branch -d feature-arrays
# Output: Deleted branch feature-arrays (was a3f9c12)

# Force delete (even if NOT merged — be careful!)
git branch -D feature-arrays
```



-d (safe delete)

Git checks that the branch has already been merged. Refuses to delete if work would be lost.



-D (force delete)

Deletes even if unmerged — you will permanently lose those commits. Use only when certain.

Best practice: Delete feature branches after merging — keeps the repo clean and easy to navigate.

```
# Full clean-up sequence after merging
git checkout main
git merge feature-arrays
git branch -d feature-arrays
```

Merging a Branch

```
# Step 1 – Finish work on your branch and commit
git add .
git commit -m "Add matrix traversal function"
```

```
# Step 2 – Switch back to main
git checkout main
```

```
# Step 3 – Merge your feature branch in
git merge feature-arrays
```

main



⚡ Outcome A — Fast-forward

```
# main had no new commits – clean!
# Output:
Updating 7b2e841..a3f9c12
Fast-forward
 matrix.py | 12 ++++++++
```

✖ Outcome B — 3-way merge

```
# Both branches had new commits
# Git creates a merge commit:
Merge made by 'recursive' strategy
 matrix.py | 5 ++++
```

If both branches changed the same line — Git cannot decide. This is a conflict. We cover that in Part D.



Practical 3 — Branch & Merge



15 min

Step 1



Init or open repo

Use your repo from
Week 4 or run
`git init week5-repo`

Step 2



Create branch

```
git checkout -b  
feature-week5
```

Step 3



Add & commit

```
Add student.py  
git add .  
git commit -m 'Add student'
```

Step 4



Merge to main

```
git checkout main  
git merge feature-week5
```



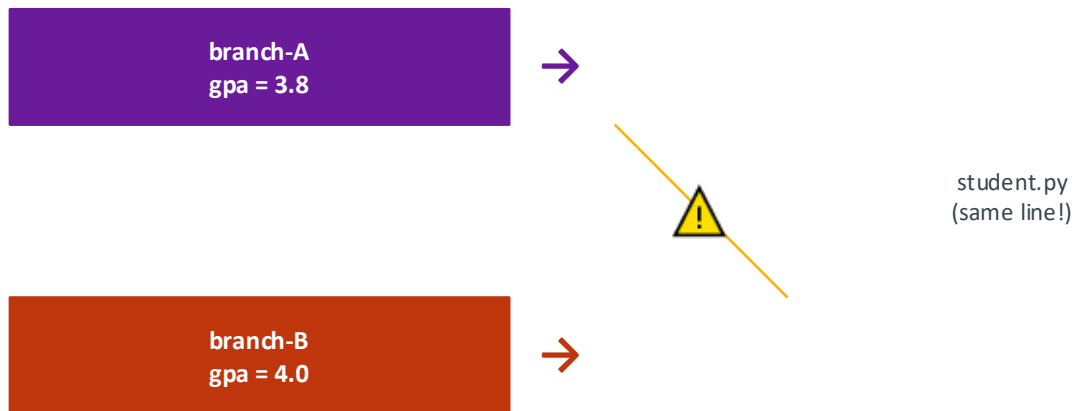
Bonus: Create a second branch feature-matrix, add matrix.py, merge that too



Run `git log --oneline` and paste the last 3 lines in chat

What is a Merge Conflict?

Conflict: When two branches change the SAME line in the SAME file — Git cannot decide which version to keep. It pauses and asks you.



When does it happen?

Two people edit the same line in the same file on different branches



Git pauses the merge

Git writes conflict markers into the file and waits for a human decision



You decide

Pick your version, their version, or combine both — then stage and commit

Reading Conflict Markers

```
<<<<<< HEAD
@dataclass
class Student:
    name: str
    age: int
    gpa: float
```

```
=====
@dataclass
class Student:
    name: str
    age: int
    gpa: float
    department: str
>>>>>> feature-department
```

YOUR version
(HEAD / current branch)

divider

THEIR version
(branch-department)

<<<<<< HEAD

Start of YOUR version — everything down to the divider is your code

=====

The divider — separates your version from the incoming version

>>>>>> branch

End of INCOMING version — this is the code from the other branch

Three Ways to Resolve a Conflict

Option 1 — Keep Yours

```
@dataclass
class Student:
    name: str
    age: int
    gpa: float
# Remove all <<< == >>> markers
```

Your original version. Their addition is discarded.

Option 2 — Keep Theirs

```
@dataclass
class Student:
    name: str
    age: int
    gpa: float
    department: str
```

Their new field is accepted. Your original is replaced.

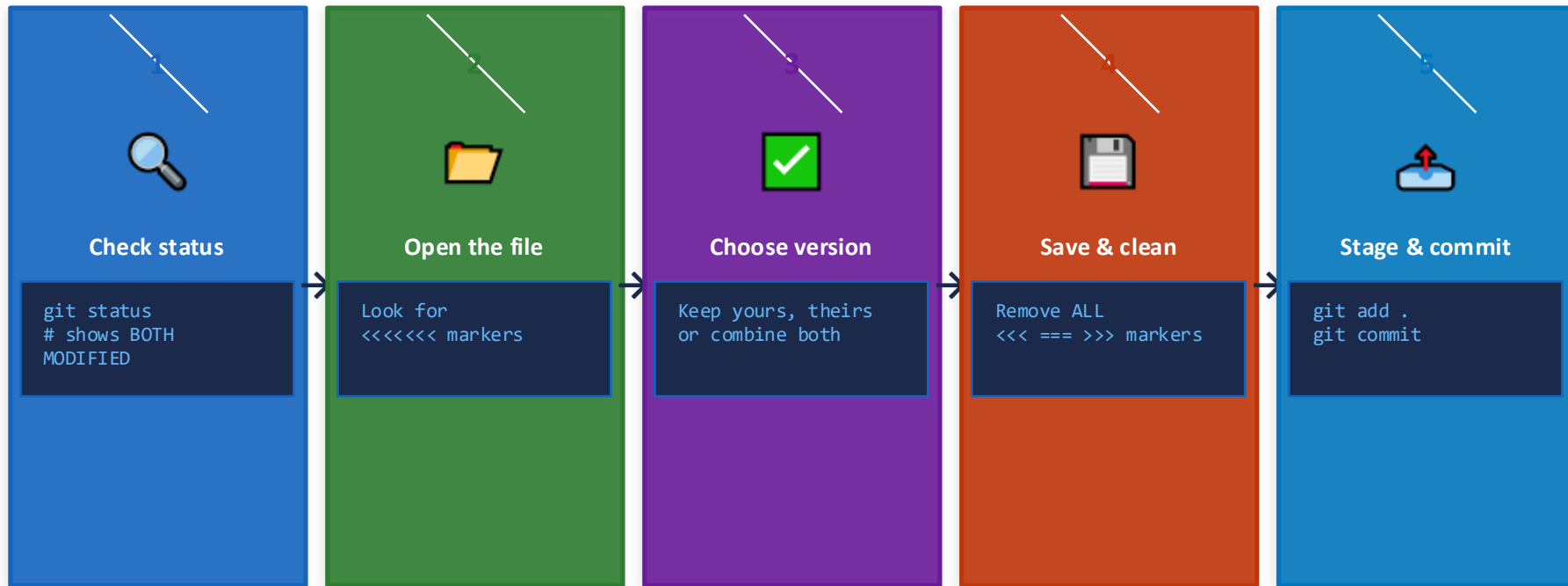
Option 3 — Combine Both

```
@dataclass
class Student:
    name: str
    age: int
    gpa: float
    department: str # ← kept their
addition
```

Best of both — keep your style AND their new field.

 Rule: NO <<<<<<, =====, or >>>>>> must remain in the file after resolution — Git treats them as syntax errors.

Resolving a Conflict — 5 Steps



💡 VS Code tip

VS Code highlights conflict markers with `Accept Current Change` / `Accept Incoming Change` buttons — click once to resolve without touching the markers manually.

Preventing Conflicts — Best Practices



Pull before you push

Always run `git pull` before starting work. You get the latest version and reduce divergence.



Work in separate files

Divide responsibilities — if you own `student.py`, your teammate owns `matrix.py`.



Communicate first

Tell teammates which file you are editing before you start. A quick message prevents 80% of conflicts.



Short-lived branches

Merge often — don't let your branch diverge for days. The longer you wait, the more conflicts you get.

In a well-organised team, most merges are conflict-free because everyone follows these four rules.



Practical 4 — Create & Resolve a Conflict



15 min



Create two branches

```
git checkout -b branch-A  
git checkout main  
git checkout -b branch-B
```



Edit the same line

On BOTH branches:
Change the gpa value
in student.py



Merge branch-A first

```
git checkout main  
git merge branch-A  
✅ Succeeds cleanly
```



Merge branch-B next

```
git merge branch-B  
❌ Conflict!  
Git pauses here
```



Resolve the conflict

Open student.py
Choose the right gpa
Remove all markers
git add . && git commit

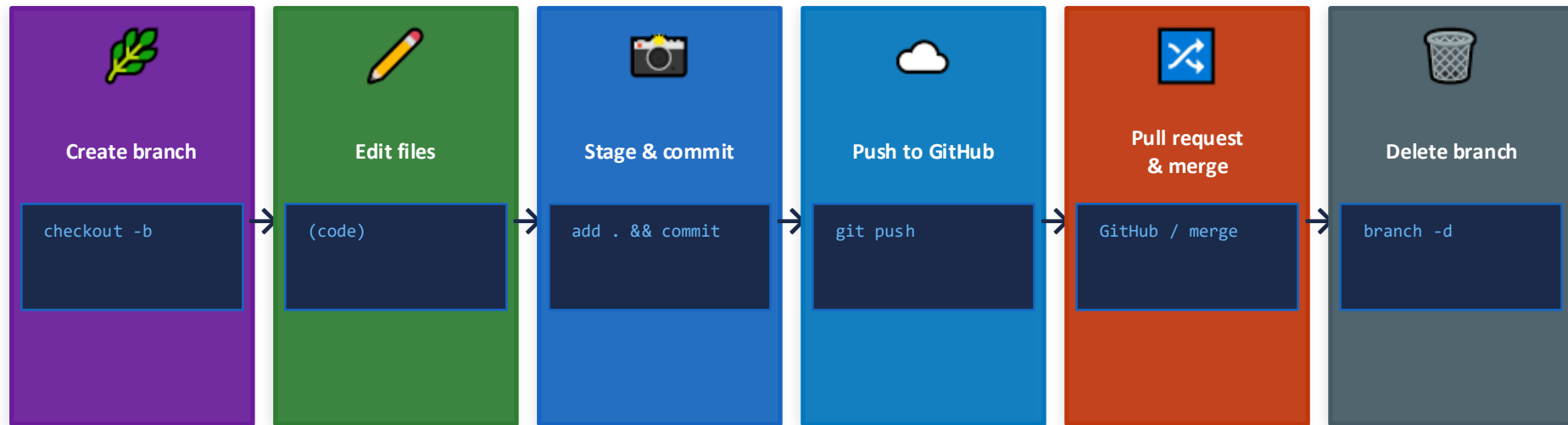


Bonus: Try resolving using VS Code's conflict UI — click Accept buttons



Screenshot your conflict markers before resolving and share in chat

Git Workflow — The Full Picture



```
git checkout -b feature → edit → git add . → git commit -m 'msg' → git push → merge → git branch -d feature
```

This is the real-world GitHub Flow used in every software team — from startups to large enterprises.

Week 5 — What We Covered

2D Arrays

Grid storage, [row][col] access, nested loop traversal — value-based and index-based

Structures

Group mixed fields with @dataclass. Create objects, access with dot notation

Arrays of Structs

Combine arrays + structures. students[i].field — the foundation of data tables

Git Branching

Parallel timelines. Create with checkout -b, switch freely, keep main clean

Merging

Fast-forward for clean merges, 3-way merge when both branches diverged

Conflict Resolution

Read <<< === >>> markers, choose version, remove markers, stage and commit

Next Week: Pointers — understanding memory addresses and how arrays really work under the hood